**FACULTY OF ENGINEERING AND BASIC SCIENCES**
**ACADEMIC PROGRAM: DATA ENGINEERING AND ARTIFICIAL INTELLIGENCE**

**COURSE: ETL (G01)**
**ETL-Project: Final Delivery**

## 1. Objectives

The final delivery integrates all the concepts and tools learned throughout the ETL course. Students will design and implement a complete data pipeline that extracts, transforms, and loads data from multiple sources, builds a dimensional model, and creates both static and real-time visualizations.

By the end of this project, you should have implemented a reproducible and automated data system capable of:

- Extracting data periodically from different sources.
- Transforming and storing it in a relational database.
- Running automated data quality checks.
- Generating KPIs and visualizations from validated data.
- Implementing a streaming ETL component that processes events and delivers meaningful low-latency analytical results.

## 2. What is Expected

The project consists of seven main components, described below. This is an incremental final delivery: refine, integrate, and extend the components developed in previous submissions rather than rebuilding the solution from scratch.

### *2.1 Requirements Definition (Refined)*

- The purpose of this stage is to clearly define the objectives, data needs, and expected outcomes of the data analysis project before building the pipeline. This step should answer "Why are we building this pipeline?" and "What insights should the data deliver?"
- Consider all the data you have and the recommendations of the past deliveries.
- ***In the end, you must show that the objectives set have been achieved.***
- Distinguish batch analytical requirements from low-latency analytical or operational requirements, and justify the value of the streaming component.

### *2.2 Data Sources*

- Use the data sources selected in previous assignments:
  - Static dataset(s): (e.g., CSV file, database).
  - A dynamic source obtained through an API.
- The API supports periodic batch extraction. Separately document how streaming events or metric updates are generated and why they require lower latency.

### *2.3 Exploratory Data Analysis (EDA) (Refined)*

- Perform EDA on all the data to understand their structure and quality.
- Document insights from the EDA that will guide your transformations and data cleaning.
- You can do the EDA in a Jupyter Notebook.
- Explain how EDA findings informed the final cleaning, transformation, and data quality rules.

### *2.4 ETL (Batch Processing using Airflow) (Refined)*

Create an Apache Airflow DAG that automates the ETL process:

- Extract: Obtain data from both sources (dataset and API).
- Transform: Clean, integrate, and prepare the data according to the requirements of your dimensional model.
- Data Quality Validation using Great Expectations: Implement problem-relevant validation suites, record results, and prevent data that fails critical checks from reaching the final Data Warehouse tables. Critical failures must be visible in Airflow.
- Load: Store the transformed data into a Data Warehouse implementing the dimensional model (fact and dimension tables).
- Provide evidence of a successful DAG execution, including quality validation results and loaded fact/dimension tables.

### *2.5 Batch Analytical Dashboard (Refined)*

- Using the Data Warehouse loaded in Section 2.4, create a batch analytical dashboard in a BI tool (e.g., Power BI, Looker Studio, or Tableau).
- The dashboard should connect to your database.
- The KPIs and visualizations must address the refined project objectives and provide business-relevant historical insights.

### *2.6 Streaming Component*

- Implement a Python Kafka producer that publishes new records or meaningful metric updates derived from the Data Warehouse fact table. Avoid repeatedly publishing unchanged KPI values without analytical justification.
- Explain how events or metric updates are generated, how often they are published, and why processing them with low latency adds value. Historical records may be replayed to simulate event arrival if clearly identified as a simulation.
- Implement a Python Kafka consumer that subscribes to the topic, validates incoming events using a basic event contract, and updates at least three relevant streaming metrics. The contract must specify required fields, data types, an event identifier, a timestamp, and the selected event key.
- Demonstrate the consumer behavior when an invalid event arrives or when the consumer is temporarily stopped; explain the observed behavior and limitations.

### *2.7 Real-Time Dashboard*

- Build a dashboard that displays at least three streaming metrics updated after new events are processed. The consumer may load results into a continuously updated database table or use another justified integration mechanism compatible with the visualization tool.
- ***Demonstrate how the dashboard changes when new events are published, explain the consumer-to-dashboard connection and update interval, and distinguish it from the historical analytical dashboard.***

**3. Deliverables**

The **only required deliverable is a NEW GitHub repository** containing the complete and integrated final version of the project.

**Important:** The repositories submitted for Delivery 1 and Delivery 2 must remain available and unchanged. Do not delete, overwrite, or replace them.

The repository must include all the source code, configuration files, database scripts, architecture diagrams, visualizations, and other resources required to understand, execute, and verify the project.

**README.md — Mandatory Technical Documentation**

The README.md must serve as the main technical documentation of the project and include:

1. **Project Overview and Objectives:** Description of the problem, refined objectives, expected outcomes, and distinction between batch and streaming requirements.
2. **Data Sources and EDA:** Description of the datasets and API, data profiling results, relevant visualizations, and how EDA findings influenced cleaning and transformation decisions.
3. **System Architecture:** Updated architecture diagram illustrating the integration of data sources, Airflow, Great Expectations, Data Warehouse, Kafka, consumers, and both dashboards.
4. **Batch ETL Pipeline:** Description of the extraction, transformation, validation, and loading processes, including the Airflow DAG structure and execution sequence.
5. **Data Quality:** Great Expectations validation rules, results, and expected behavior when critical checks fail.
6. **Data Warehouse:** Dimensional model, fact and dimension tables, database schema, and loading strategy.
7. **Batch Analytical Dashboard:** Description of the KPIs, visualizations, and main analytical insights.
8. **Streaming Pipeline:** Kafka producer and consumer implementation, event generation strategy, topic configuration, event contract, validation rules, and at least three streaming metrics.
9. **Real-Time Dashboard:** Description of the consumer-to-dashboard integration, visualization mechanism, update frequency, and main insights.
10. **Execution and Reproducibility:** Software requirements, environment setup, configuration, commands to start services, and step-by-step instructions to execute and verify the complete system.
11. **Functional Evidence:** Screenshots or other evidence demonstrating successful Airflow execution, data quality checks, Data Warehouse loading, Kafka producer/consumer

operation, streaming metric updates, and dashboard functionality. Include evidence of at least one invalid-event or consumer-interruption experiment.

12. **Engineering Decisions and Limitations:** Explanation of important technical decisions, assumptions, known limitations, and relevant reliability or security considerations.

**Additional Requirements**

- Include a .gitignore file and a requirements.txt or equivalent dependency specification.
- Do not publish credentials, API keys, passwords, or other sensitive information.
- Include the necessary dashboard files, configuration files, and SQL scripts, or provide accessible links when appropriate.
- Organize the repository using a clear and consistent folder structure.
- Ensure that the instructions in the README.md allow another person to reproduce the implementation.

**No separate technical report or PDF document is required.** All technical documentation and evidence must be included in the GitHub repository, primarily through its README.md.

**4. Technology Stack**

| Layer | Tool/Technology | Purpose |
|---|---|---|
| Orchestration | Apache Airflow | Schedule and monitor batch ETL workflows |
| Event streaming | Apache Kafka | Publish, retain and distribute events |
| Data sources | Static dataset(s), public API | Provide historical and periodically updated data |
| Data storage | PostgreSQL / MySQL | Store the dimensional model and processed streaming data |
| Quality checks | Great Expectations | Validate data before loading final tables |
| Transformation | Python (Pandas) | Clean, integrate and transform data |
| Visualization | Power BI, Looker Studio, etc. | Historical and low-latency analytics |

| Layer | Tool/Technology | Purpose |
|---|---|---|
| Versioning | Git, GitHub, Markdown | Version control and project documentation |

## 5. Diagram

Figure 1 illustrates the recommended integration of batch ETL and streaming processing. The architecture may be adapted to the project requirements with appropriate justification.

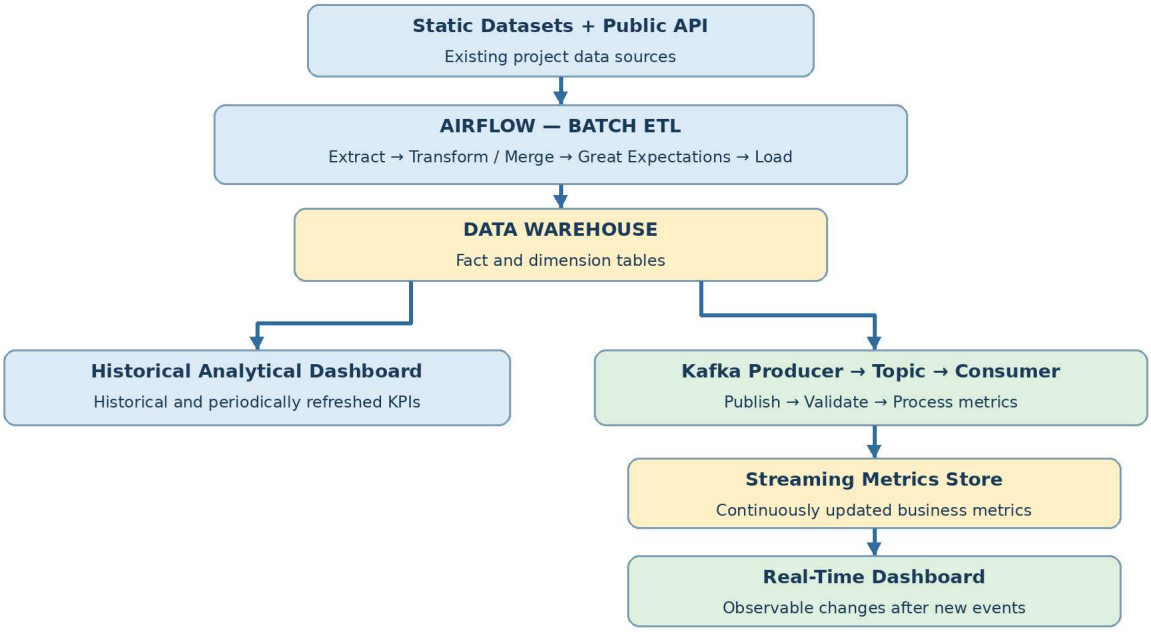



**Figure 1. Integrated Batch and Streaming Architecture**

## 6. Evaluation

Evaluation is based on the integrated, working project and its demonstrated results. The following criteria total 5.0 points.

| Assessment criterion | Weight |
|---|---|
| Requirements refinement and EDA | 0.5 |
| Batch ETL: Airflow integration and automation | 0.8 |
| Data quality: Great Expectations and failure behavior | 0.6 |
| Dimensional model and Data Warehouse loading | 0.6 |
| Historical analytical dashboard | 0.5 |
| Kafka streaming pipeline and event processing | 0.8 |
| Real-time dashboard and observable metric updates | 0.7 |
| GitHub Repository, README Documentation and Reproducibility | 0.5 |
| **TOTAL** | **5.0** |

During the final presentation, demonstrate the main pipeline components and explain how new events affect the streaming metrics and dashboard. Evidence should reflect actual execution, not screenshots of code alone.